Reporte

Nombres: Equipo 5 **Rodrigo Noguera Pérez** **Andrik Hasiel Pérez Limón** **Erick Antonio Paez Estrada**	**Matrículas**: 2136238 2118898 2120761
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural del Sistema (Tecno Stock)
Fecha: 17/10/2026	

Bibliografía (Formato APA)

FIME UANL. (2026). Caso 05: TecnoStock.

Noguera Pérez, R., Pérez Limón, A. H., & Paez Estrada, E. A. (2026). BRD de TecnoStock [Documento del curso Componentes de lógica de negocio].

Noguera Pérez, R., Pérez Limón, A. H., & Paez Estrada, E. A. (2026). User Stories de TecnoStock [Documento del curso Componentes de lógica de negocio].

Evans, E. (2004). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design. John Wiley & Sons.

Richards, M., & Ford, N. (2020). Fundamentals of software architecture: An engineering approach. O'Reilly Media.

Propósito de la Actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Este documento demuestra cómo el equipo pasó de los requerimientos de la

Unidad 2 a una arquitectura que separa dominio, aplicación e infraestructura para el proceso central de "Venta, inventario, apartados y devoluciones" en Tecno Stock.

La actividad no consistió en pedirle a una IA una solución terminada: en cada etapa se solicitaron alternativas, se contrastaron contra el caso real y se documentó por qué una propuesta corresponde o no al negocio (ver bitácora de IA, sección Vibe Coding).

Introducción

La transformación de un documento de requerimientos de negocio y casos de uso en una arquitectura de software robusta y mantenible exige un análisis riguroso de las reglas, restricciones y comportamientos esperados del sistema. El presente informe detalla la conceptualización arquitectural para el ecosistema tecnológico de Tecno Stock, un retail de electrónica cuyas operaciones sufren actualmente vulnerabilidades críticas: inconsistencias entre el sistema y el inventario físico, ventas accidentales de productos apartados y dificultad para rastrear números de serie.

Para abordar este desafío ingenieril y solucionar la "fuga de mercancía", el desarrollo del diseño se ha estructurado siguiendo estrictamente las etapas de la definición arquitectural orientada al dominio estipuladas en la rúbrica de evaluación.

Etapa 1: Preparar el caso

El establecimiento de una arquitectura de software coherente exige delimitar los flujos transaccionales que poseen el mayor impacto estratégico. Para Tecno Stock, el objetivo primario es doble: garantizar la trazabilidad de la "Unidad Exacta" a través de números de serie y asegurar matemáticamente que la mercancía apartada no se sobrevenda.

Se han seleccionado seis casos de uso arquitecturalmente significativos que abarcan flujos normales, flujos alternativos asíncronos y flujos de excepción.

ID	Caso de uso	Actor	Objetivo/Entrada	Reglas Relacionadas (Evidencia U2)	Resultado Esperado
CU-01	Consulta de disponibilidad global (flujo normal)	Vendedor	Verificar stock en la cadena ***productoSKU***	BRD 5.1; US-01, US-02 (Consultar inventario local y en otras tiendas)	Sistema muestra existencias físicas globales; permite ofrecer alternativas al cliente.

Reporte

CU-02	Creación de apartado y cálculo de anticipo (flujo normal)	Vendedor/ cajero	Asegurar reserva ***clienteId***, ***productos***	BRD 5.1; US-08 (Cobro mínimo del 20%, límite de 5 artículos, reserva temporal 15 min)	Reserva temporal creada; el sistema exige cobro; inventario bloqueado
CU-03	Liquidación final de apartado (Flujo normal)	Cajero	Liquidar deuda pendiente ***apartadoId***	BPMN; US-11 (Pago del saldo restante antes del vencimiento de 30 días)	Ticket de compra generado con código QR; transacción cerrada
CU-04	Vencimiento de apartado y monedero (flujo excepción)	Sistema	Liberar stock retenido ***apartadoId***	US-12 (Liberar inventario retenido tras 30 días sin liquidar; generar monedero).	Stock disponible para venta general; anticipo convertido en saldo a favor.
CU-05	Entrega física con escaneo de número de serie (Flujo normal)	Almacen	Registrar salida ***ticketQR***, ***numeroSerie***	BRD 5.1; US-15, US-16 (Asignar número de serie único; rastreo individual)	Sistema asocia el número de serie al ticket y registra la salida física
CU-06	Autorización y Envío de Transferencia.	Supervisor / Almacén	Trasladar mercancía.	US-14 (Emisión), US-15 (Recepción).	Mercancía descontada del origen y sumada al destino.

Etapa 2: Elección del Patron de Lógica de Negocio

Para determinar el patrón más adecuado para Tecno Stock, se ha elaborado la siguiente matriz comparativa evaluando cada opción a través del prisma de las exigencias del caso de estudio documentado en la Unidad 2.

Criterio del caso Tecno Stock	**Transaction Script**	**Table Module**	**Domain Model**	**Evidencia de Trazabilidad U2**
Complejidad de las reglas	**Inadecuado:** Forzaría duplicar la lógica de cálculo (anticipo 20%) y temporizadores (15 min y 30 días) en múltiples scripts procedimentales	**Parcial:** Centraliza reglas por tabla, pero dificulta severamente flujos que cruzan productos, apartados y monederos.	**Óptimo:** Encapsula las políticas de anticipos y vencimientos dentro de entidades ricas, aislando la complejidad.	E-01: BRD 5.1 y US-08 (Límites de 5 artículos, cobros del 20%, reserva de 15 min).
Interacción entre conceptos	**Deficiente:** Requiere consultas SQL ad-hoc constantes para validar si un producto ya está apartado antes de venderlo.	**Deficiente:** Fomenta dependencias circulares entre el módulo de Ventas y el módulo de Inventario Seriado.	**Óptimo:** Facilita la colaboración en memoria (Un ***Inventario*** transfiere unidades a un ***Apartado*** atómicamente).	E-02: US-04, US-05, US-06 (Interacción logística entre Sucursal Origen, Destino y Almacén)

Reporte

Estados e invariantes	**Peligroso:** Objetos anémicos. Un script de venta directa podría saltarse la validación de apartado y cometer sobreventa.	**Regular:** Protege a nivel de esquema SQL, pero no representa semánticamente el ciclo de vida comercial.	**Óptimo:** Protege férreamente la regla crítica: Un producto reservado jamás debe venderse a otro cliente.	E-03: BRD Sección 5.2 Restricciones (Un producto reservado/apartado no debe venderse bajo ninguna circunstancia)
Mantenibilidad (criterio adicional)	**Bajo:** Alto riesgo de "código espagueti" al incorporar las reglas de devoluciones y mermas	**Medio:** Depende de un mapeo relacional directo que ralentiza refactorizar lógicas.	**Alto:** Aísla el núcleo. Permite someter las reglas de vencimiento a pruebas unitarias sin tocar la Base de Datos.	E-04: BRD Sección 3.2 Problema (Vulnerabilidades actuales: inconsistencia de criterios de devolución)

Ventajas, riesgos y consecuencias por alternativa

- **Transaction Script**. Ventaja: Rapidez de desarrollo inicial.
  Riesgo: Duplicación de validaciones (ej. validar si un pago cubre el 20%). Consecuencia: Inconsistencias graves que permitirían apartar mercancía con anticipos insuficientes.

- **Table Module**. Ventaja: Más estructurado que scripts sueltos.
  Riesgo: Lógicas complejas de vencimiento a los 30 días no tienen un lugar natural. Consecuencia: La lógica de coordinación terminaría mezclada en la capa de aplicación.

- **Domain Model**. Ventaja: Encapsula reglas e invariantes donde pertenecen. Riesgo: Curva de aprendizaje más pronunciada para identificar Aggregates. Consecuencia: Mayor inversión en la Etapa 3, a cambio de asegurar que la existencia del sistema cuadre matemáticamente con el físico.

Decisión Argumentada

Con base en la evaluación cualitativa presentada y respaldada por la evidencia, la decisión central establece la adopción irrestricta de Domain Model.

La fundamentación radica profundamente en la naturaleza altamente condicional del dominio de Tecno Stock. Resolver el problema de "existencia de sistema no coincide

con inventario físico" (BRD Sección 3.2) y evitar la "venta accidental de productos apartados" exige garantizar a nivel memoria que una transacción proteja sus estados. El Domain Model permite que el objeto ***Apartado*** autoevalúe sus tiempos (US-09: 15 minutos, US-12: 30 días) y altere su ciclo de vida autónomamente, emitiendo *Domain Events* que detonen la generación del monedero electrónico sin depender de un flujo lineal.

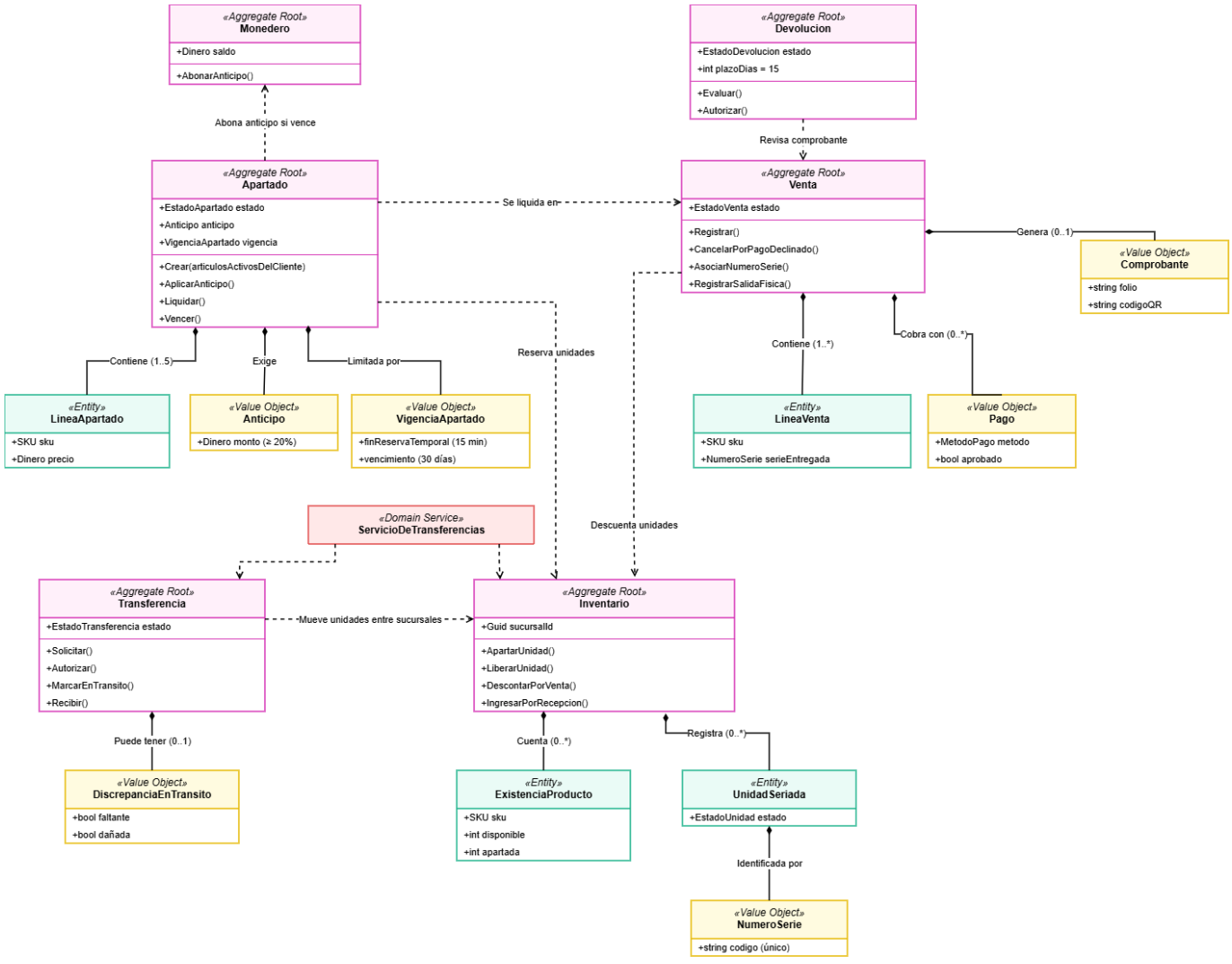
Etapa 3: Construcción del Mapa de Dominio

La justificación teórica del Domain Model requiere el uso extensivo del Diseño Orientado al Dominio (DDD). A continuación, se delimitan las responsabilidades.

Elemento	**Respuesta del equipo para Tecno Stock**
Lenguaje ubicuo	Apartado (reserva de productos con anticipo), Anticipo (pago mínimo del 20%), Reserva Temporal (15 min para pagar el anticipo), Vencimiento (30 días para liquidar), Monedero (saldo a favor por apartado vencido), Unidad Exacta (pieza física con número de serie), Transferencia, En Tránsito y Comprobante (ticket con QR). Son los mismos términos del glosario y se usan sin sinónimos en el BRD y el código. “Transacción” se unificó como Venta, y “reservar” y “apartar” como Apartado.
Entidades	Tienen identidad y ciclo de vida propio: **Apartado**, **Inventario** (uno por Sucursal), **UnidadSeriada**, **Venta**, **Transferencia**, **Monedero**, **Devolucion**, más las entidades internas LineaApartado, ExistenciaProducto y LineaVenta. Cliente, Producto, Sucursal y Empleado se manejan por id como datos de catálogo y quedan fuera de este mapa.
Value Objects	Definidos por sus valores, inmutables y sin identidad: Dinero, Anticipo (monto + regla del 20%), VigenciaApartado (fin de la reserva temporal de 15 min y vencimiento a 30 días), NumeroSerie, SKU, Pago, MetodoPago, Comprobante (folio + código QR), DiscrepanciaEnTransito y los estados tipados (EstadoApartado, EstadoUnidad, EstadoTransferencia, EstadoVenta).
Aggregates	Seis, cada uno con una sola raíz como puerta de entrada: **Apartado**, **Inventario**, **Venta**, **Transferencia**, **Monedero** y **Devolucion**. Los cuatro primeros cubren los 5 casos de uso de la Etapa 1; Monedero se activa en CU-04 y Devolucion se modela a nivel básico.
Domain Services	**ServicioDeTransferencias**: lógica apátrida que coordina a Transferencia con el Inventario de la sucursal origen y del destino (descontar al enviar, ingresar al recibir), porque la regla no pertenece a un único Aggregate.
Domain Events	ReservaTemporalExpirada, ApartadoConfirmado, ApartadoLiquidado, ApartadoVencido, SaldoAFavorGenerado, VentaRegistrada, TransaccionCancelada, SalidaFisicaRegistrada, TransferenciaSolicitada, TransferenciaAutorizada, TransferenciaEnviada, TransferenciaRecibida, DiscrepanciaEnTransitoRegistrada y DevolucionAutorizada. Son hechos históricos e inmutables, nombrados en pasado, que otros componentes escuchan; como, por ejemplo, ApartadoVencido libera el stock en Inventario y abona el anticipo al Monedero.

3.1 Diagrama del Mapa de Dominio

Cada raíz (rosa) es la única puerta de entrada de su Aggregate y las partes que contiene aparecen debajo de ella. Verde: entidades internas; amarillo: Value Objects; rojo: Domain Service. Las líneas con rombo indican composición (contiene); las flechas punteadas entre Aggregates indican referencia por id o reacción a un evento, nunca un objeto compartido.



3.2 Fichas de Aggregates y fronteras transaccionales

Aggregate	Raíz	Responsabilidades	Invariantes protegidas	Eventos emitidos
Apartado	Apartado (contiene LineaApartado, Anticipo y VigenciaApartado)	Crear la reserva temporal, calcular y aplicar el anticipo, confirmar, liquidar y vencer. Evalúa sus propios plazos recibiendo la hora actual como parámetro.	1) Máximo 5 artículos. 2) Anticipo mínimo del 20% del total. 3) La reserva temporal expira a los 15 min sin pago. 4) Solo se liquida antes del vencimiento (30 días). 5) Transiciones válidas: ReservaTemporal → Confirmado → Liquidado o Vencido.	ReservaTemporalExpirada, ApartadoConfirmado, ApartadoLiquidado, ApartadoVencido
Inventario	Inventario por Sucursal (contiene ExistenciaProducto y UnidadSeriada)	Apartar, liberar, descontar e ingresar unidades; responder la disponibilidad local y global.	1) Un producto apartado o reservado no puede venderse a otro cliente. 2) Un número de serie es único y pertenece a una sola sucursal. 3) La existencia nunca es negativa y cuadra con sus unidades. 4) Una unidad En Tránsito no es vendible en ninguna sucursal.	(los consume y reacciona; registra cada cambio de estado de unidad para trazabilidad)

Reporte

Venta	Venta (contiene LineaVenta, Pago y Comprobante)	Registrar el cobro, cancelar si el pago es declinado, emitir ticket con QR y registrar la salida física con número de serie.	1) El comprobante con QR solo existe con pago aprobado. 2) Un pago declinado cancela la venta sin tocar inventario. 3) Un artículo seriado exige el número de serie exacto de su SKU. 4) La salida física se registra una sola vez.	VentaRegistrada, TransaccionCancelada, SalidaFisicaRegistrada
Transferencia	Transferencia (contiene DiscrepanciaEnTransito)	Llevar el ciclo solicitud → autorización → envío → recepción o discrepancia.	1) Solo un Supervisor autoriza. 2) Solo una transferencia Autorizada pasa a En Tránsito. 3) Solo se ingresa si llega completa y sin daños; con discrepancia queda detenida. 4) Origen y destino son sucursales distintas.	TransferenciaSolicitada, TransferenciaAutorizada, TransferenciaEnviada, TransferenciaRecibida, DiscrepanciaEnTransitoRegistrada
Monedero	Monedero por cliente	Abonar el anticipo de un apartado vencido y descontar saldo.	1) El saldo nunca es negativo. 2) Un apartado vencido abona una sola vez (idempotencia por apartadoId).	SaldoAFavorGenerado
Devolucion	Devolucion (plazo de 15 días como atributo)	Validar comprobante, estado físico y plazo; escalar al	1) Solo dentro del plazo de 15 días.	DevolucionAutorizada

		Supervisor cuando hay excepción.	2) Requiere comprobante válido. 3) Las excepciones requieren autorización explícita del Supervisor.	

3.3 Análisis de decisiones dudosas y descartadas

Decisión Descartada: Fusionar Apartado y Venta en un solo Aggregate "Transacción".

- **Planteamiento**: El glosario usa "Transacción" y "Venta" como sinónimos, por lo que se consideró modelar un único Aggregate que cubriera tanto el apartado como la venta directa, para simplificar el modelo y evitar un evento entre ambos.
- **Justificación del Rechazo**: Sus ciclos de vida y reglas son distintos: el Apartado vive hasta 30 días, maneja temporizadores (15 min y 30 días) y puede terminar en monedero, mientras que la Venta es inmediata y termina en la entrega con número de serie. Fusionarlos mezclaría invariantes sin relación y agrandaría la unidad de consistencia. Se mantienen separados y se conectan mediante el evento ApartadoLiquidado.

Decisión Descartada: Independizar UnidadSeriada como Aggregate Raíz.

- **Planteamiento**: Se debatió que cada unidad con número de serie fuera su propio Aggregate, para bloquear solo la unidad que se aparta o se vende y no cargar todo el Inventario de la sucursal.
- **Justificación del Rechazo**: La regla "un producto reservado jamás se vende a otro cliente" y la cuadratura de existencias involucran varias unidades a la vez, por lo que necesitan una raíz común que las proteja. Al encapsular UnidadSeriada dentro de Inventario aceptamos cargar más datos al operar, que se mitiga cargando solo los SKU afectados, pero garantizamos que es imposible vender una unidad apartada y que la existencia siempre cuadra con sus unidades.

Decisión Descartada: Proteger el límite de 5 artículos solo dentro del Aggregate Apartado.

- **Planteamiento**: Como el BPMN expresa la regla como “máximo 5 artículos por cliente” y la US-08 solo dice “no excede el límite de 5 artículos”, se asumió que el límite aplicaba por apartado y que el Aggregate Apartado podía garantizarlo por sí solo como invariante.
- **Justificación del Rechazo**: El stakeholder confirmó que el límite es de 5 artículos por persona en total, sumando todos sus apartados activos. Como cada Apartado es un Aggregate distinto y ninguno conoce a los demás, no puede garantizar esa regla por sí solo. La regla se mantiene en el dominio: el Application Service obtiene del repositorio cuántos artículos activos tiene ya el cliente (en reserva temporal o confirmados) y se lo pasa a Apartado.Crear(), que rechaza la creación si el total supera 5. Para evitar que dos apartados simultáneos del mismo cliente rebasen el límite, la consulta y el guardado ocurren dentro del mismo Unit of Work.

Etapa 4: Diseñar la Service Layer

La Service Layer no concentra reglas de negocio; actúa como el director de orquesta que coordina entidades y repositorios.

Servicio	Método/Caso de uso	Entrada	Colaboradores	Salida	Reglas que delega al dominio
Apartado Service	IniciarApartado (CU-02)	***clienteId, listaProductos***	IInventarioRepo, IApartadoRepo, Apartado	ApartadoDTO (con monto min 20%)	Consulta primero cuántos artículos activos tiene el cliente y se lo delega al Aggregate Apartado, que valida el límite de 5 por cliente y calcula matemáticamente el anticipo del 20%
Apartado Service	ProcesarVencimientos (CU-04)	***ahora***	IApartadoRepo, IInventarioRepo, IMonederoRepo, Apartado	ResultDTO (apartados vencidos procesados)	Delega al Aggregate Apartado validar que esté Confirmado y que hayan pasado 30 días (Vencer). Ante el evento ApartadoVencido, Inventario libera las unidades y Monedero abona el anticipo, de forma idempotente por apartadoId

Reporte

Caja Service	LiquidarApartado (CU-03)	***apartadoId, montoPago***	IApartadoRepo, Apartado	TicketCompraDTO	Delega a Apartado verificar que la liquidación ocurra estrictamente antes del vencimiento (30 días)
Almacen Service	RegistrarSalidaFisica (CU-05)	***ticketId, serieId***	IVentaRepo, Venta	SalidaConfirmadaDTO	Delega al dominio la asociación de la "Unidad Exacta" (Número de serie) al ticket correspondiente
Transferencia Service	AutorizarEnvio	***transferenciaId, supervisorId***	ITransferenciaRepo, ServicioDeTransferencias	ResultDTO	Delega a la validación de dominios asegurar que solo cambie a estado "En tránsito" si se cumplen las reglas.

Revisión Obligatoria

- **¿El nombre del método expresa una acción del negocio?** Sí: ***IniciarApartado***, ***LiquidarApartado*** y describen el lenguaje de los actores (Vendedor, Cajero, Almacén).
- **¿El método coordina o contiene decisiones?** Se revisó: Ningún Application Service calcula el 20% ni evalúa límites de tiempo; todos delegan en el Aggregate (***Apartado.AplicarAnticipo()***).
- **¿El servicio depende de interfaces?** Sí,
- exclusivamente de interfaces de repositorio (más el Unit of Work): ***IApartadoRepository***, ***IInventarioRepository***, ***IVentaRepository***, ***ITransferenciaRepository*** y ***IMonederoRepository***, sin ORMs.
- **¿La operación tiene inicio y fin transaccional?** Sí, al liquidar un saldo (CU-03), el Unit of Work asegura que se marque como pagado y se genere el QR en un solo bloque atómico.

Etapa 5: Separar persistencia y transacciones

Aquí definimos los Repositorios guiados por los Aggregate Roots (Apartado e Inventario) para proteger la lógica de la persistencia estructural.

Repository	**Necesidad del dominio**	**Métodos propuestos**	**Objetos que recupera o guarda**
IInventarioRepository	Controlar los saldos físicos de cada tienda,	***ObtenerPorSucursal(id)***, ***Guardar(Inventario)***	***Inventario*** (Aggregate Root con stock local)

	aplicando deducciones o sumando transferencias recibidas		
IApartadoRepository	Cargar un apartado en memoria para registrar su liquidación (US-11) o ejecutar la validación nocturna de vencimiento (US-12) y contar los artículos activos del cliente para validar el límite de 5 al iniciar un apartado	***ObtenerPorId(id)*, *Guardar(Apartado)*, *ListarVencidos()* ContarArticulosActivos(clienteId)**	***Apartado*** (Aggregate Root completo con fechas de reserva)
IVentaRepository	Cargar la venta a partir del ticket con QR para registrar la salida física y asociar el número de serie del producto entregado (US-15, US-16)	***ObtenerPorTicket(ticketId), Guardar(Venta)***	***Venta*** *(Aggregate Root completo con sus líneas y comprobante)*
ITransferenciaRepository	Cargar una transferencia para autorizar su envío, registrar su salida y recepción o reportar discrepancias en tránsito (US-03 a US-07)	***ObtenerPorId(id), Guardar(Transferencia)***	***Transferencia*** *(Aggregate Root completo)*
IMonederoRepository	Recuperar el monedero del cliente para abonar el anticipo de un apartado vencido (US-12)	***ObtenerPorCliente(clienteId), Guardar(Monedero)***	***Monedero*** *(Aggregate Root con su saldo a favor)*

Propuesta de Unit of Work: El alcance transaccional debe evitar catástrofes en el inventario.

Caso*:* **US-13** y **US-14** (Venta directa y cancelación de transacción). Cuando el Cajero registra una venta y envía la petición de cobro, inicia el Unit of Work. Si el método de pago es declinado, la capa de aplicación dispara un Rollback automático de toda la transacción. Esto asegura que la base de datos retroceda y "los productos permanezcan en stock general" (US-14), cumpliendo con la regla de negocio de no alterar inventarios por ventas fallidas.

Etapa 6: Separar Commands y Queries (CQRS Básico)

Dado que Tecno Stock sufre de lentitud y bloqueos de base de datos debido a que 5 sucursales realizan consultas simultáneas de stock (Problema del BRD), aplicar CQRS resuelve el cuello de botella.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Query	ConsultarDisponibilidadGlobal Query	***sku***	ConsultaStock Handler	ListaDisponibilidad DTO	Alta concurrencia (US-01/US-02). Lectura masiva plana que no bloquea la tabla transaccional. No requiere instanciar el Aggregate ***Inventario***.
Query	EscanearTicketParaEntrega Query	***codigoQR***	ConsultarTicket Handler	DetalleTicket DTO	Almacén escanea QR (US-15). Es de solo lectura para visualizar en pantalla los artículos pagados.
Command	CrearReservaTemporal Command	***clienteId***, ***arts***	CrearReserva Handler	CommandResult	Modifica inventario y genera un timer crítico (15 min) (US-08). Ejecuta fuerte lógica de dominio.
Command	AutorizarTransferencia Command	***solicId***, ***supId***	AutorizarTransf Handler	CommandResult	Alteración de estado (De "Disponible" a "En tránsito"). Requiere verificación de roles de Supervisor (US-04)

Etapa 7: Revisar la propuesta con SOLID

7.1 Diseño sometido a revisión

Capa	Componentes revisados
Dominio	**Dominio (Aggregates): Venta, Apartado, Inventario, Transferencia, Monedero, Devolucion.** **Dominio (Value Objects): NumeroDeSerie, MetodoDePago, VigenciaApartado, Anticipo, Comprobante.**
Aplicación	**CajaService, AlmacenService, ApartadoService, TransferenciaService, TicketService, y consultas de disponibilidad (US-01, US-02).**
Infraestructura	**IVentaRepository, IApartadoRepository, IInventarioRepository, ITransferenciaRepository, IMonederoRepository, IProcesadorDePago, UnitOfWork.**

7.2 Revisión por principio

Principio	Pregunta de revisión	Evidencia encontrada en TecnoStock
S	**¿Cada clase o servicio tiene una razón principal para cambiar?**	**CajaService solo coordina la venta. El ticket QR lo genera un TicketService independiente y la existencia la protege el Aggregate Inventario.**
O	**¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?**	**El límite de 5 artículos y el 20% de pago mínimo viven en los Value Objects Anticipo y VigenciaApartado. Los pagos se evalúan mediante la abstracción IProcesadorDePago.**
L	**¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?**	**Se evitó que ProductoConSerie herede de Producto, ya que un subtipo seriado rechazaría ventas "por cantidad" (ej. cables) rompiendo el contrato. Se usa el atributo requiereNumeroSerie.**
I	**¿Cada consumidor depende solo de las operaciones que utiliza?**	**Se separó la escritura (IInventarioRepository para almacén) de las consultas de disponibilidad en piso (CQRS de lectura), evitando que el vendedor dependa de métodos de almacén.**

Principio	Pregunta de revisión	Evidencia encontrada en TecnoStock
D	**¿La aplicación y el dominio dependen de abstracciones?**	**Los Application Services (CajaService, ApartadoService) dependen de abstracciones genéricas (repositorios), sin referencias a Entity Framework ni a terminales bancarias físicas.**

7.3 Hallazgos registrados

#	Hallazgo	Evidencia	Impacto	Cambio realizado
1	VentaService mezclaba cobro, inventario y ticket QR (riesgo S)	CajaService validaba inventario, cobraba y armaba el ticket QR en el mismo método.	Cambiar el formato del ticket obligaría a modificar la lógica de inventarios.	Se extrajo TicketService, invocado por el evento VentaRegistrada.
2	Reglas de apartado hardcodeadas (Riesgo O)	Las reglas del 20% de anticipo y 5 artículos estaban escritas directo en ApartadoService.	Modificar las políticas de la tienda obligaría a reescribir el servicio.	Se movieron las reglas a los Value Objects Anticipo y VigenciaApartado.
3	Métodos de pago con switch (Riesgo O)	Un switch gigante evaluaba si era efectivo, tarjeta, etc.	Agregar un nuevo banco alteraría el código central de ventas.	Se delegó a la abstracción IProcesadorDePago.
4	Herencia de Producto seríado (Riesgo L)	Subclase ProductoConSerie propuesta para la US-16.	Rompería la iteración de la caja al cobrar productos genéricos por cantidad.	Se usa el atributo requiereNumeroSerie y AlmacenService asocia la serie.
5	Repositorio monolítico (Riesgo I)	Una sola interfaz gigante manejaba desde transferencias hasta lecturas de piso.	Los vendedores tendrían acceso a métodos de almacén que no usan.	Se separaron las lecturas con un modelo CQRS.
6	Dependencia de SQL (Riesgo D)	Código amarrado directamente a la base de datos y la terminal física.	Imposibilidad de correr pruebas o cambiar de tecnología.	Los Application Services ahora dependen estrictamente de Interfaces (IRepository).

#	Hallazgo	Evidencia	Impacto	Cambio realizado
7	Estados de la transferencia y autorización bien ubicados (confirma decisión)	**La transición de estados y que solo un Supervisor autorice transferencias, se valida dentro del Aggregate Transferencia (US-03 a US-07).**	**Si vivieran en el servicio, otro flujo podría saltarse la autorización o ingresar mercancía con discrepancia al inventario.**	**Sin cambio. TransferenciaService solo coordina y se documenta la decisión como validada.**

Etapa 8: Representar y documentar

Se presentan tres vistas de la misma arquitectura: un diagrama de clases por capas, dos diagramas de secuencia para los flujos más complejos y los Architecture Decision Records (ADR) que justifican las decisiones.

Diagrama de clases

El diagrama separa las tres capas definidas en las etapas anteriores: **Aplicación** (los Application Services de la Etapa 4), **Dominio** (Aggregates, entidades, Value Objects y Domain Service del mapa de la Etapa 3) e **Infraestructura** (interfaces de repositorio y Unit of Work de la Etapa 5). Las flechas punteadas son dependencias: los servicios dependen de interfaces, nunca de una implementación concreta, y cada repositorio persiste un Aggregate Root completo. Las Queries de la Etapa 6 leen un modelo plano y no instancian Aggregates, por lo que no aparecen aquí.

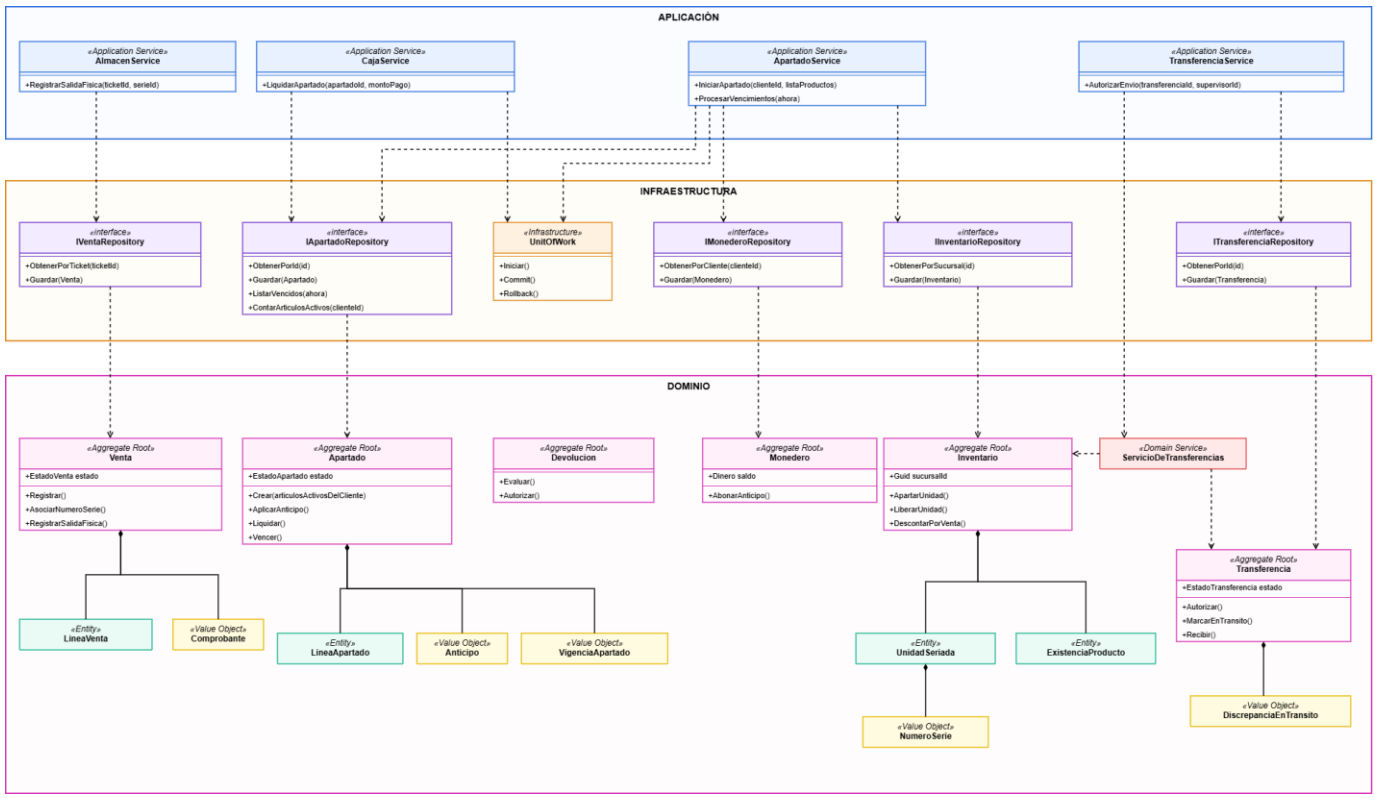


Diagrama de secuencia 1 (flujo principal)

CU-02: creación de un apartado. Se valida el límite de 5 artículos por cliente, se crea la reserva temporal de 15 minutos con el anticipo mínimo del 20% y se bloquea el inventario, con rechazo por límite excedido o por falta de existencia:

Reporte

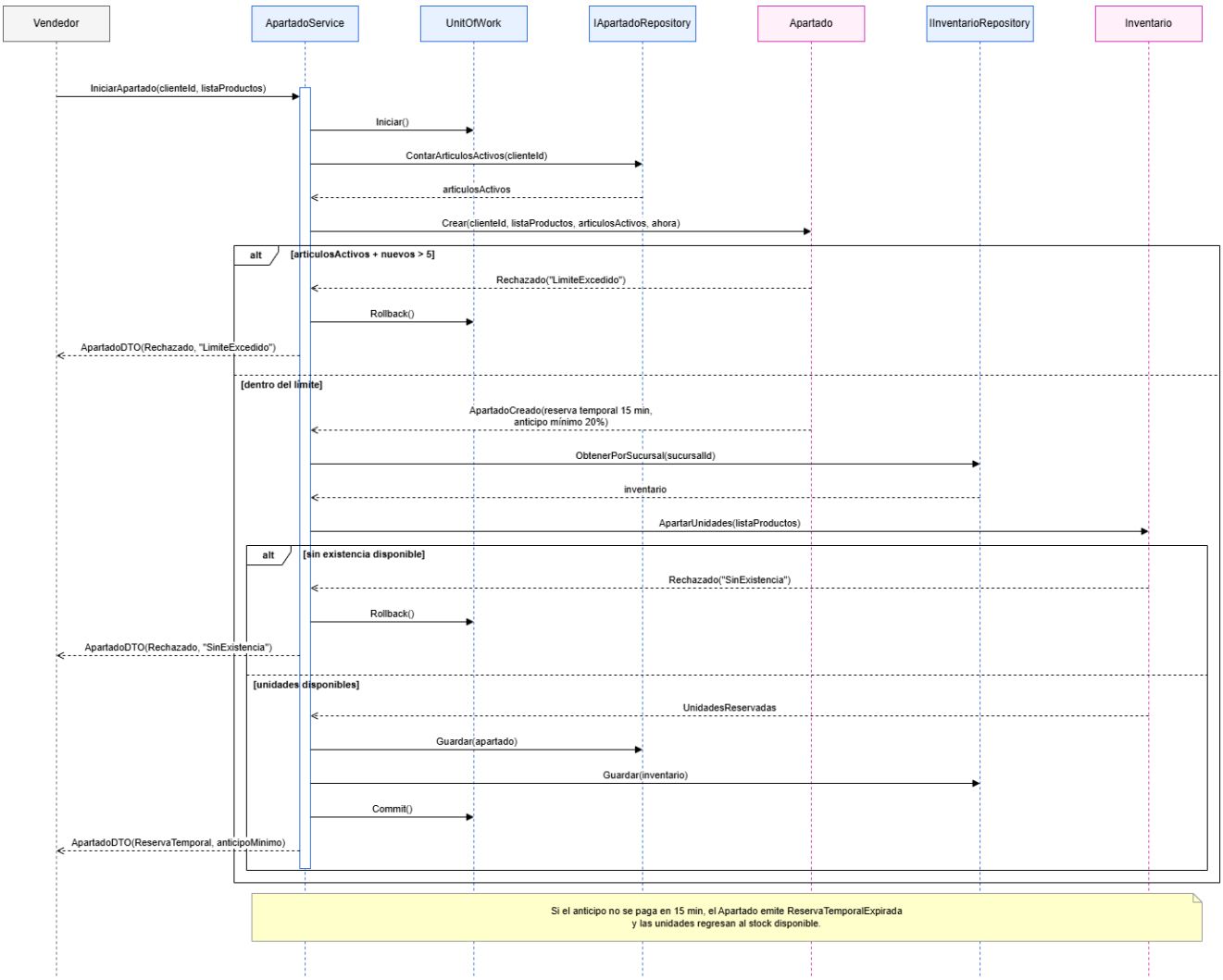
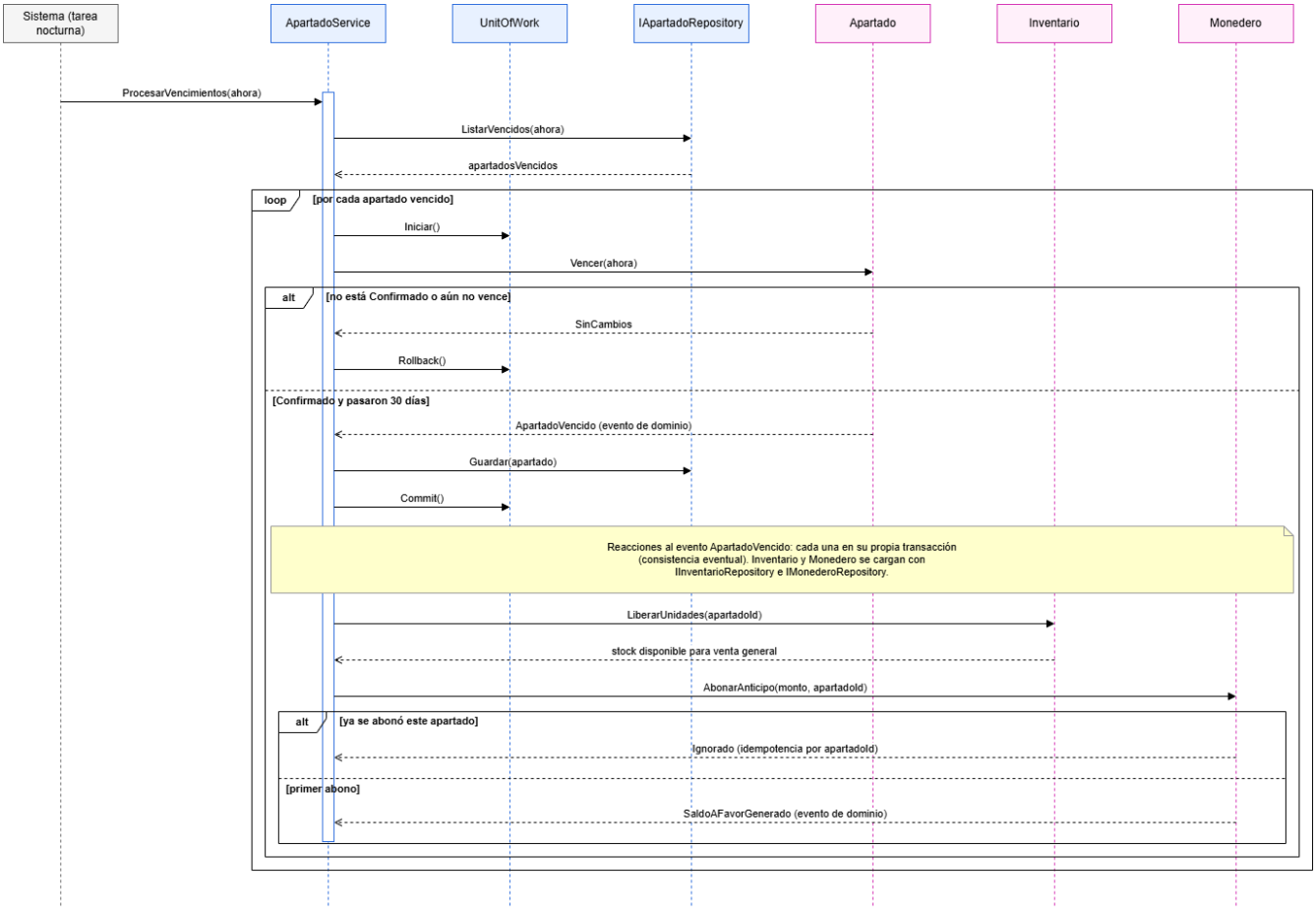


Diagrama de secuencia 2 (flujo de excepción)

CU-04: vencimiento de un apartado a los 30 días sin liquidar. El evento ApartadoVencido provoca que el stock retenido vuelva a estar disponible y que el anticipo se abone al monedero del cliente:



Nos decidimos por estos dos flujos porque son los más complejos: cruzan tres Aggregates (Apartado, Inventario y Monedero), involucran reglas de tiempo y de límite, y muestran la diferencia entre la consistencia inmediata dentro de un Unit of Work y la consistencia eventual por eventos de dominio.

Architecture Decisions Records

ADR-01 — Adoptar Domain Model como patrón de lógica de negocio

Estado	Aceptada
Contexto	El proceso de ventas, apartados e inventario de Tecno Stock tiene reglas numerosas e interdependientes con manejo de estado (anticipo del 20%, reserva temporal de 15 min, vencimiento a 30 días, números de serie), confirmadas en el BRD 5.1 y 5.2 y en las US-08 a US-12.
Alternativas	Transaction Script (duplicaría validaciones entre scripts y permitiría vender mercancía apartada) y Table Module (no representa el ciclo de vida del apartado ni las reglas que cruzan Ventas e Inventario).
Decisión	Se adopta Domain Model para todo el núcleo de apartados, inventario, ventas y transferencias; no se combina con otro patrón.
Consecuencias	Mayor inversión inicial en el mapa de dominio (Etapa 3); las reglas de anticipo y vencimiento se prueban sin base de datos; trabajo posterior: mantener pequeños los Aggregates, sobre todo Inventario, al crecer el catálogo.
Trazabilidad	BRD 3.2 y 5.2; US-08, US-09, US-12

ADR-02 — Usar Unit of Work para operaciones compuestas

Estado	Aceptada
Contexto	Iniciar un apartado modifica Apartado e Inventario (bloqueo de unidades) y una venta directa puede fallar por pago declinado. Ninguna de esas operaciones puede dejar unidades bloqueadas o descontadas a medias (BRD 5.2; US-14).
Alternativas	Guardar cada Aggregate por separado sin transacción explícita, o usar consistencia eventual entre Apartado e Inventario; ambas dejan una ventana en la que un producto apartado podría venderse a otro cliente.
Decisión	Se introduce un Unit of Work que inicia al comenzar el Application Service y hace Commit solo cuando todos los Aggregates modificados se guardaron; ante cualquier rechazo o error, Rollback completo. Las reacciones a eventos posteriores (liberación por vencimiento, abono al monedero) se ejecutan cada una en su propia transacción.
Consecuencias	Mayor consistencia de datos y ninguna unidad bloqueada por operaciones fallidas; una capa adicional de infraestructura por mantener; trabajo posterior: definir el manejo de reintentos ante fallos de red o de base de datos.
Trazabilidad	US-08, US-13, US-14; BRD 5.2

ADR-03 — Aplicar CQRS básico solo en consultas de alto volumen

Estado	**Aceptada**
Contexto	Cinco sucursales consultan el stock de forma simultánea y la lentitud y bloqueos de la base de datos son un problema documentado en el BRD. ConsultarDisponibilidadGlobalQuery y EscanearTicketParaEntregaQuery son de solo lectura, a diferencia de CrearReservaTemporalCommand y AutorizarTransferenciaCommand, que ejecutan lógica de dominio.
Alternativas	Aplicar CQRS a todo el sistema (más complejidad sin beneficio claro) o no aplicarlo en ningún lado (consultas lentas si cada lectura carga el Aggregate Inventario completo).
Decisión	Se separan Queries de Commands únicamente para las consultas de alto volumen; las Queries leen un modelo plano y no instancian Aggregates.
Consecuencias	Consultas más rápidas que no bloquean las tablas transaccionales; dos rutas de código por mantener solo donde se justifica; trabajo posterior: definir qué tan actualizado debe estar el modelo de lectura frente a un apartado recién creado.
Trazabilidad	US-01, US-02, US-04, US-08, US-15; BRD (problema de lentitud)

ADR-04 — Validar el límite de 5 artículos en el dominio con el conteo aportado por el servicio

Estado	**Aceptada**
Contexto	El stakeholder confirmó que un cliente no puede apartar más de 5 artículos en total, sumando todos sus apartados activos. Cada Apartado es un Aggregate distinto y ninguno conoce a los demás.
Alternativas	Validar el límite solo dentro de Apartado (no puede ver los otros apartados del cliente) o validarlo en el Application Service (sacaría una regla de negocio del dominio).
Decisión	El Application Service consulta IApartadoRepository.ContarArticulosActivos(clienteId) y pasa el resultado a Apartado.Crear(), que rechaza la creación si el total supera 5. La regla permanece en el dominio y el servicio solo aporta el dato.
Consecuencias	La regla se prueba sin base de datos pasando el conteo como parámetro; depende del Unit of Work (ADR-02) para que dos apartados simultáneos del mismo cliente no rebasen el límite; se cuentan los artículos en reserva temporal o confirmados.
Trazabilidad	US-08; BPMN; BRD 5.1

Vibe Coding con IA

El equipo usó IA como copiloto para contrastar alternativas de diseño de TecnoStock, revisar el diseño con SOLID y producir borradores de la validación. La responsabilidad y la autoría de cada decisión son del Equipo 5: cada resultado se comparó contra el Caso 05 y las user stories US-01 a US-16 antes de integrarse a este documento, y lo que no tenía respaldo en esas user stories se descartó o se registró como duda pendiente.

Ciclo de trabajo seguido

Paso	Acción	Evidencia en TecnoStock
1	Preparar contexto	Se seleccionaron del Caso 05 los apartados §2 (proceso y métodos de pago), §4 (transferencias y Supervisor) y §5 (diferencia entre sistema e inventario físico), junto con US-01 a US-16 y el glosario del equipo (Vendedor, Cajero, Supervisor, Personal de Almacén, Apartado, Monedero, Ticket, Número de serie).
2	Solicitar propuesta	Se pidió a la IA una alternativa concreta por etapa (patrón, dominio, servicios, CQRS, SOLID), exigiendo que declarara sus supuestos y sin pedir que reescribiera todo el diseño.
3	Comparar	Cada respuesta se contrastó con las reglas de las historias (20% de anticipo, límite de 5 artículos, 15 minutos de reserva temporal, 30 días de vigencia, autorización del Supervisor) y con los conceptos de la unidad.
4	Corregir	Se ajustaron nombres al glosario, se eliminaron elementos sin respaldo en las historias (por ejemplo, devoluciones) y se movieron reglas del servicio hacia el Aggregate correspondiente.
5	Validar	Se revisó trazabilidad contra US-01 a US-16, consistencia entre diagramas y el comportamiento esperado de cada caso de uso (Etapa 9, secciones 9.1 a 9.3).
6	Registrar	Se guardó el resumen del prompt, la respuesta útil, el error o supuesto detectado y la decisión humana en la bitácora siguiente.

Reporte

Bitácora obligatoria de IA

It.	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
1	Elegir el patrón de lógica de negocio (Etapa 2)	Comparar Transaction Script, Table Module y Domain Model para TecnoStock con base en apartados, transferencias con autorización y entrega con número de serie, declarando supuestos.	Tabla comparativa por criterio con consecuencias de cada alternativa; recomienda Domain Model por las reglas interdependientes entre apartado, inventario y monedero.	Presentó las historias como operaciones simples y sugirió Transaction Script para la venta directa, sin considerar la reserva de existencia ni el rollback por pago declinado (US-14).	Se adoptó Domain Model (ADR-01) para todo el proceso, sin mezclar patrones.
2	Construir el mapa de dominio (Etapa 3)	Proponer candidatos a entidades, Value Objects, Aggregates y eventos a partir del glosario y US-01 a US-16, indicando evidencia y dudas.	Aggregates Venta, Apartado, InventarioDeSucursal, Transferencia y MonederoElectronico; Value Objects NumeroDeSerie, MetodoDePago, Vigencia y PoliticaDeApartado; eventos VentaRegistrada y ApartadoVencido.	Propuso además un Aggregate de Devolución y un Cliente con historial de compras; ninguna historia entre US-01 y US-16 los respalda.	Se excluyeron del diseño. Las devoluciones quedaron como duda pendiente (sección 9.4).

Reporte

It.	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
3	Diseñar Service Layer, Repository y Unit of Work (Etapas 4 y 5)	Proponer firmas de Application Services y repositorios a partir de los casos de uso, separando coordinación de reglas del dominio.	VentaService, EntregaService, ApartadoService, TransferenciaService y TicketService; interfaces de repositorio e IProcesadorDePago; Unit of Work por operación.	Nombres técnicos (ActualizarInventario, GuardarVenta) y validación del 20% de anticipo dentro de ApartadoService.	Se renombraron a acciones del negocio y la regla pasó a Apartado con PoliticaDeApartado.
4	Separar Commands y Queries (Etapa 6)	Identificar qué operaciones cambian estado y cuáles solo consultan, evitando separar donde no aporte.	US-01 y US-02 como Queries con modelo de lectura; registrar venta, crear apartado y autorizar transferencia como Commands.	Sugería bases de datos separadas de lectura y escritura para todo el sistema, sin respaldo en el caso.	CQRS básico solo para las consultas de disponibilidad, sin bases separadas.
5	Revisar el diseño de TecnoStock con SOLID (Etapa 7)	Analizar el diseño de Service Layer, Repository y Aggregates de TecnoStock y señalar posibles violaciones SOLID, sin reescribir todo.	7 hallazgos con evidencia, impacto y cambio sugerido por principio.	La primera versión incluía devoluciones y valores no definidos que las historias de usuario no respaldan. Se contrastó con US-01 a US-16 y se corrigió.	Se aceptaron los 7 hallazgos tras ajustarlos a las US; las devoluciones quedaron fuera del diseño.

Reporte

It.	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
6	Validar la propuesta (Etapa 9)	Recorrer CU-01 a CU-05 sobre el diseño indicando quién recibe la solicitud, dónde se valida cada regla, qué cambia y qué se devuelve, y señalar contradicciones con las historias.	Tabla de recorrido por caso de uso (9.2) y lista de dudas para el stakeholder.	Dio por hecho que el inventario baja al cobrar, pero US-16 registra la salida física al entregar; también asumió que el límite de 5 artículos era por cliente.	Se documentó como supuesto en 9.4: la venta compromete la existencia y la entrega registra la salida; el límite de 5 queda pendiente de validar.
7	Revisar congruencia de los Casos de Uso con la Matriz de Trazabilidad.	Revisa que los Casos de Uso del diseño cuadren exactamente con la Matriz de Trazabilidad.	Se detectó un desfase en la numeración (CU-01 a CU-06) entre el reporte de la Etapa 9 y la Matriz final.	El reporte listaba la "Venta Directa" como CU-01, desplazando la numeración oficial de los apartados y transferencias.	Se reestructuró la tabla 9.2 para coincidir 1 a 1 con la Matriz (CU-01 a CU-06), dejando la Venta Directa como un flujo adicional.

Etapa 9: Validar la propuesta

Se recorrió cada caso de uso de Tecno Stock sobre los diagramas y el diseño, verificando qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persiste y qué resultado se devuelve.

9.1 Validaciones

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. Los servicios, Aggregates y la política de apartado se relacionan con al menos una historia entre US-01 y US-16 y con un problema del Caso §4 o §5 (ver matriz 9.3).

Validación	Pregunta	Resultado del equipo
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí. Se usan los términos del glosario y de las historias (Vendedor, Cajero, Supervisor, Personal de Almacén, Apartado, Reserva temporal, Anticipo, Monedero, Ticket, Número de serie, En tránsito, Discrepancia en Tránsito) sin sinónimos nuevos.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Sí. La existencia disponible vive en InventarioDeSucursal, las reglas de anticipo, límite y tiempos en Apartado con PoliticaDeApartado, y los estados y la autorización en Transferencia. Ningún Application Service decide reglas del negocio.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. Los participantes de ambos diagramas de secuencia (servicios, Aggregates y repositorios) existen como clases en el diagrama de clases, con los mismos nombres.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Sí. Venta, Apartado, InventarioDeSucursal, Transferencia y MonederoElectronico no referencian repositorios ni ORM; solo los Application Services dependen de las interfaces.
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí. El Unit of Work inicia en el Application Service y hace Commit al guardar todos los Aggregates modificados. Ante un error hay Rollback completo, por ejemplo un pago declinado no deja cambios en el inventario (US-14).
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí. La bitácora registra las 6 iteraciones (Etapas 2 a 9) con el error o supuesto detectado y la decisión del equipo; ningún resultado de la IA se integró sin contrastarlo con US-01 a US-16.

Decisión consciente: en CU-01 a CU-06 el Unit of Work abarca más de un Aggregate. Se acepta porque el problema central del caso es la diferencia entre el sistema y el inventario físico (Caso §5), y una consistencia eventual dejaría ventanas donde se vende un producto apartado.

9.2 Recorrido de cada caso de uso

Caso de uso	Recibe la solicitud	Dónde se valida cada regla	Qué cambia y cómo se persiste	Resultado devuelto
CU-01 Consulta de disponibilidad global US-01, US-02. Actor: Vendedor	ConsultarDisponibilidadGlobalQuery	N/A (Solo lectura). La consulta no modifica el estado del sistema.	Ninguno. Lee directo del modelo plano sin instanciar Aggregates ni Unit of Work.	Lista de existencias en las 5 sucursales.
CU-02 Creación de apartado y cálculo de anticipo US-08, US-09, US-10. Actor: Vendedor / Cajero	ApartadoService.CrearApartado() y ConfirmarApartado()	Apartado con PoliticaDeApartado: límite de 5 artículos, anticipo del 20% y pago dentro de 15 min.	Se crea Apartado. InventarioDeSucursal reserva la existencia. Al confirmar, vence en 30 días. Usa Unit of Work.	ApartadoDTO. Rechazos: LimiteExcedido, AnticipoInsuficiente, ReservaExpirada.

Reporte

Caso de uso	Recibe la solicitud	Dónde se valida cada regla	Qué cambia y cómo se persiste	Resultado devuelto
CU-03 Liquidación final de apartado US-11. Actor: Cajero	CajaService.LiquidarApartado()	Apartado.EstaVencido(): Valida que el pago total ocurra antes de los 30 días.	Apartado pasa a Liquidado y Venta genera el Comprobante QR. Usa Unit of Work.	TicketDTO con código QR. Rechazo: ApartadoVencido.
CU-04 Vencimiento de apartado y monedero US-12. Actor: Sistema	ApartadoService.ProcesarVencimientos()	Apartado.EstaVencido(ahora): 30 días sin liquidar.	Apartado pasa a vencido, InventarioDeSucursal libera stock, MonederoElectronico abona anticipo. Usa Unit of Work.	Lista de apartados liberados y montos abonados.
CU-05 Entrega física con númer	EntregaService.EscanearTicket(qr) y EntregarMercancia()	Venta: ticket pagado. Producto: requiereNumeroSerie. InventarioDeSu	Venta asocia el NumeroDeSerie e InventarioDeSucursa	EntregaDTO. Rechazos: TicketNoPagado, SerieNoCorresponde.

Reporte

Caso de uso	Recibe la solicitud	Dónde se valida cada regla	Qué cambia y cómo se persiste	Resultado devuelto
o de serie US-15, US-16. Actor: Almacén		cursal: unidad disponible.	l registra la salida física. Usa Unit of Work.	
CU-06 Transferencia entre sucursales US-03 a US-07. Actor: Supervisor / Almacén	TransferenciaService: Solicitar, Autorizar, Enviar y Recibir Transferencia	Transferencia valida cambios de estado y autorización de Supervisor. Almacén verifica que llegue completa.	Origen descuenta y pasa a "En tránsito". Destino ingresa si está completa, o pasa a "Discrepancia". Usa Unit of Work por paso.	TransferenciaDTO. Rechazos: NoAutorizada, DiscrepanciaEnTransito.
Flujo Adicional: Venta directa y cobro	VentaService.RegistrarVenta()	InventarioDeSucursal: existencia disponible. Venta: cobro aprobado.	Se crea Venta y se compromete existencia. Si el pago es	TicketDTO con código QR. Rechazos: PagoDeclinado,

Caso de uso	Recibe la solicitud	Dónde se valida cada regla	Qué cambia y cómo se persiste	Resultado devuelto
US-13, US-14. Actor: Cajero			declinado hay Rollback (stock no se afecta).	ProductoApartado.

Las consultas de disponibilidad (US-01 y US-02) no cambian estado, por eso se atienden con el modelo de lectura y no pasan por un Aggregate ni por el Unit of Work.

9.4 Supuestos y dudas pendientes de validar con el stakeholder

Se detectaron al validar el diseño contra las historias de usuario. No se inventó ninguna regla; se registran como hallazgos de la Unidad 2.

- Cuándo disminuye el inventario: el Caso 2 dice que, al finalizar la venta, pero US-16 registra la salida física al entregar con número de serie. El diseño asume que la venta compromete la existencia y la entrega registra la salida física.
- El límite de 5 artículos (US-08): si es por apartado o por cliente con apartados activos.
- Qué puede hacer el cliente con el saldo del monedero y si tiene vigencia (US-12 solo define cómo se genera).
- Cómo se resuelve una Discrepancia en Tránsito (reposición, merma o reclamo); US-07 solo detiene el ingreso al inventario.
- Devoluciones: el Caso 2, 4 y 5 las mencionan, pero US-01 a US-16 no las cubren. Si entran en el alcance, faltan historias, reglas y un servicio de devolución.
- Qué otros casos, además de las transferencias, exigen autorización de Supervisor (Caso 4).

Matriz de trazabilidad

La matriz parte de cada necesidad o regla del negocio (BRD y Caso 05) y sigue su camino hasta la historia de usuario, el caso de uso, el componente de la arquitectura y la decisión que la resuelve. Corresponde al diseño final del informe de arquitectura del equipo (Domain Model, Service Layer, Repository, Unit of Work y CQRS básico).

A. **Matriz principal: necesidad de negocio → caso de uso → regla → stories → componente → decisión**

Reporte

Necesidad de negocio / Problema	Caso de uso	Reglas de Negocio Relacionadas	User stories (US)	Componentes de arquitectura	Decisiones (ADR)
Clientes que acumulan más apartados de los permitidos	CU-02	Máximo 5 artículos por persona, sumando todos sus apartados activos (confirmado por el stakeholder).	US-08	ApartadoService; IApartadoRepository.Contar ArticulosActivos; Apartado.Crear()	ADR-04; ADR-02
Anticipo perdido cuando el apartado vence	CU-04	A los 30 días sin liquidar se libera el stock y el anticipo pasa al monedero, una sola vez.	US-12	ApartadoService.ProcesarVencimientos; Aggregate Monedero (idempotente por apartadoId); IMonederoRepository	ADR-02; hallazgo 1 (SOLID: S)
Dificultad para rastrear números de serie (BRD 3.2)	CU-05	El número de serie es único, corresponde al SKU del ticket y se asocia a la venta al entregar.	US-15, US-16	AlmacenService; Venta.AsociarNumeroSerie(); UnidadSeriada y NumeroSerie; IVentaRepository	ADR-01; hallazgo 3 (SOLID: L)
Lentitud y bloqueos por consultas simultáneas de las cinco sucursales (BRD 3.2)	CU-01	La consulta de existencias local y en otras sucursales no modifica el estado del sistema.	US-01, US-02	ConsultarDisponibilidadGlobalQuery y su Handler; modelo de lectura plano	ADR-03; hallazgo 4 (SOLID: I)
Entrega de mercancía sin comprobar el pago	CU-03, CU-05	El ticket con código QR solo existe con pago aprobado o apartado liquidado.	US-11, US-13, US-15	CajaService; Comprobante (Venta); EscanearTicketParaEntregaQuery	ADR-02; ADR-03

Transferencias entre sucursales sin control ni trazabilidad de discrepancias	CU-06	Solo un Supervisor autoriza; solo una transferencia Autorizada pasa a En Tránsito; con discrepancia no ingresa al inventario.	US-03, US-04, US-05, US-06, US-07	TransferenciaService; Aggregate Transferencia; ServicioDeTransferencias	ADR-02; hallazgo 4 (SOLID: S)
Ventas registradas con pago declinado	Complemento (venta directa)	Un pago declinado cancela la transacción y los productos permanecen en stock general.	US-13, US-14	Aggregate Venta (TransaccionCancelada); UnitOfWork.Rollback()	ADR-02
Aplicación y dominio atados a la base de datos	Todos	El dominio no depende de SQL ni de un ORM; los servicios usan interfaces.	US-08, US-11, US-12, US-16	IApartadoRepository, IInventarioRepository, IVentaRepository, ITransferenciaRepository, IMonederoRepository	Hallazgos 5 y 6 (SOLID: I y D)

Reporte

Cobertura de cada caso de uso

Caso de uso	User Stories	Application Service o Query	Aggregates y eventos	Persistencia
CU-01 Consulta de disponibilidad global	US-01, US-02	ConsultarDisponibilidadGlobal Query	Ninguno (modelo de lectura plano)	Sin Unit of Work
CU-02 Creación de apartado y cálculo de anticipo	US-08, US-09, US-10	ApartadoService.Iniciar Apartado	Apartado, Inventario; ReservaTemporalExpirada	IApartadoRepository, IInventarioRepository, UnitOfWork
CU-03 Liquidación final de apartado	US-11	CajaService.Liquidar Apartado	Apartado, Venta (Comprobante); ApartadoLiquidado	IApartadoRepository, UnitOfWork
CU-04 Vencimiento de apartado y monedero	US-12	ApartadoService.Procesar Vencimientos	Apartado, Inventario, Monedero; ApartadoVencido, SaldoAFavorGenerado	IApartadoRepository, IInventarioRepository, IMonederoRepository
CU-05 Entrega física con número de serie	US-15, US-16	EscanearTicketParaEntregaQuery; AlmacenService.RegistrarSalidaFisica	Venta, Inventario; SalidaFisicaRegistrada	IVentaRepository, UnitOfWork
CU-06 Autorización y Envío de Transferencia	US-03, US-04, US-05, US-06, US-07	TransferenciaService.AutorizarEnvio	Transferencia, Inventario; TransferenciaAutorizada	ITransferenciaRepository, IInventarioRepository, UnitOfWork